BitCraft Public Mechanics Guide

This guide documents mechanics that can be confirmed from the public BitCraft server repository at `clockworklabs/BitCraftPublic`. It is written for app developers who need to understand what the public server code proves, what it only hints at, and what should not be assumed without live data.

Scope And Confidence

This document uses the public repository as the source of truth. A mechanic is listed as confirmed only where the public code shows the data model, reducer, helper function, or formula. Where the repository exposes table schemas but not the live static-data records, this guide documents the schema and logic but does not invent exact live percentages, costs, or thresholds.

Important limitation: the repository contains Rust definitions for many static-data tables, such as recipes, loot tables, item descriptions, claim technology, and resource descriptions. In this review, no committed JSON or CSV data dump for the live records was found. That means the public repository proves how the server uses those records, but often not the actual current values inside production.

Repository Map

The relevant public server code is mostly under:

- `BitCraftServer/packages/game/src/messages/components.rs` - public SpacetimeDB state tables.
- `BitCraftServer/packages/game/src/messages/static_data.rs` - static-data table schemas for items, recipes, loot, technology, resources, buildings, and more.
- `BitCraftServer/packages/game/src/game/handlers/` - reducer logic for player actions and claim actions.
- `BitCraftServer/packages/game/src/game/entities/` - helper methods on state/data types.
- `BitCraftServer/packages/game/src/agents/` - scheduled/background systems such as rent and housing income.
- `BitCraftServer/packages/global_module/src/` - global/empire-level systems, separate from normal claim-local state.

Claims And Settlement State

Claims are represented by `ClaimState`, while mutable local settlement values live in `ClaimLocalState`. The public local claim state includes supplies, maintenance, tile count, treasury, and an XP remainder used for treasury minting.

Confirmed fields:

- `ClaimState` contains `entity_id`, `owner_player_entity_id`, `owner_building_entity_id`, `name`, and `neutral` (`BitCraftServer/packages/game/src/messages/components.rs:1185-1194`).
- `ClaimLocalState` contains `supplies`, `building_maintenance`, `num_tiles`, `location`, `treasury`, and `xp_gained_since_last_coin_minting` (`BitCraftServer/packages/game/src/messages/components.rs:1207-1223`).

- `ClaimMemberState` contains the member player id, username, and inventory/build/officer/co-owner booleans (`BitCraftServer/packages/game/src/messages/components.rs:1235-1251`).

Buildings/resources are considered under a claim only when every walkable or hitbox footprint tile maps to the same claim. If a footprint is split between claims, the helper returns no claim (`BitCraftServer/packages/game/src/game/claim_helper.rs:56-84`, `BitCraftServer/packages/game/src/game/claim_helper.rs:86-112`).

Claim totem placement is constrained by distance from existing claim totems, distance from claims, and prohibited biomes (`BitCraftServer/packages/game/src/game/claim_helper.rs:313-345`). The exact distance between claims is read from `parameters_desc().version().find(&0).unwrap().min_distance_between_claims` (`BitCraftServer/packages/game/src/game/claim_helper.rs:325-329`), so the formula is visible but the live parameter value is not proven by this source review.

Treasury And Hex Coin Income

Claim Treasury Storage

Claim treasury is stored on `ClaimLocalState.treasury` as a `u32` (`BitCraftServer/packages/game/src/messages/components.rs:1217-1219`).

Crafting XP Can Mint Hex Coins Into The Claim Treasury

Confirmed formula:

```
threshold = minimum positive xp_to_mint_hex_coin from learned claim tech
total_xp = previous_xp_remainder + gained_xp
minted_hex = floor(total_xp / threshold)
new_xp_remainder = total_xp % threshold
treasury += minted_hex
```

The threshold is derived from learned claim tech:

- `ClaimTechState::min_xp_to_mint_hex_coin` scans learned tech IDs.
- It reads `claim_tech_desc.xp_to_mint_hex_coin`.
- It uses only positive values.
- If no positive value exists, it effectively disables minting by returning `u32::MAX`.

Source: `BitCraftServer/packages/game/src/game/entities/claim_tech_state.rs:40-54`.

The minting reducer then calculates `num_minted_coins = total_xp / xp_to_mint_hex_coin`, adds that amount to treasury, and stores the remainder in `xp_gained_since_last_coin_minting` (`BitCraftServer/packages/game/src/game/claim_helper.rs:374-393`).

The exact live threshold cannot be calculated from the public repository alone because the code defines `ClaimTechDesc.xp_to_mint_hex_coin`, but the live `claim_tech_desc` records are not committed here (`BitCraftServer/packages/game/src/messages/static_data.rs:2119-2135`).

How Crafting Feeds Treasury Minting

Crafting awards XP based on `experience_per_progress.quantity` and progress actions. The code:

- Calculates `base_damage` from tool power and skill power.
- Calculates `damage` from tool power multiplied by crit multiplier.
- Uses `base_damage` for XP action count.
- Uses crit-adjusted `damage` for craft progress.
- Awards XP as `ceil(experience_per_progress.quantity * experience_actions_count)`.
- If the building is in a claim, sends that same XP quantity into `mint_hex_coins`.

Source: `BitCraftServer/packages/game/src/game/handlers/player_craft/craft.rs:449-477`.

Interpretation: a crit can advance craft progress faster than the XP action count used in the same tick, because XP uses `base_damage` while craft progress uses crit-adjusted `damage` (`BitCraftServer/packages/game/src/game/handlers/player_craft/craft.rs:458-468`). This is an interpretation of the code path, not a live-balancing statement.

Manual Treasury Deposits And Withdrawals

Owner/co-owner players can deposit hex coins into the claim treasury. The code removes hex coins from the player inventory and nearby deployables, then adds the amount to `claim_local.treasury` (`BitCraftServer/packages/game/src/game/handlers/claim/claim_treasury_deposit.rs:12-35`).

Owner/co-owner players can withdraw from the treasury. The code checks treasury balance, adds hex coins to the player's inventory, then subtracts from `claim_local.treasury` (`BitCraftServer/packages/game/src/game/handlers/claim/claim_withdraw_from_treasury.rs:12-34`).

Rent And Housing Income

Rent collection can add `daily_rent` to claim treasury when rent is paid (`BitCraftServer/packages/game/src/agents/rent_collector_agent.rs:72-93`).

Player housing income can also add income from qualifying housing buildings to claim treasury. The agent looks up building descriptions, calculates `BuildingFunction::player_housing_income(&desc)`, accumulates income per claim, and updates `ClaimLocalState` (`BitCraftServer/packages/game/src/agents/player_housing_income_agent.rs:82-113`).

Barter Stall Treasury Effects

Barter stall transactions can spend from or add to town treasury:

- If offered coins are needed and the stall inventory lacks enough, the code pays partly or fully from claim treasury (`BitCraftServer/packages/game/src/game/handlers/player_trade/barter_stall_order_accept.rs:304-331`).
- If gained items include coins, those coins are added to town treasury (`BitCraftServer/packages/game/src/game/handlers/player_trade/barter_stall_order_accept.rs:350-364`).

This is a barter stall path, not proof that normal auction listings tax or fund the claim treasury.

XP And Level Formula

The public code contains the profession XP level formula in `ExperienceStack`.

Constants:

- `MAX_LEVEL = 125`
- `MULTIPLIER = 10`
- `LEVEL_ONE_EXPERIENCE = 52`
- `growth_rate = 2.0 ^ 0.158`

Formula:

```
experience_for_level(level) =
  floor(52 * ((growth_rate^level - growth_rate) / (growth_rate^2 - growth_rate))) * 10
```

`experience_until_next_level(level)` subtracts the current level threshold from the next level threshold. `level_for_experience(experience)` increments from level 1 until the next level threshold is greater than the current XP, or max level is reached.

Source: `BitCraftServer/packages/game/src/game/entities/experience_stack.rs:6-43`.

Crafting And Production

Crafting recipes are described by `CraftingRecipeDesc`, which includes:

- building requirement,
- level requirements,
- tool requirements,
- consumed item stacks,
- required claim tech,
- XP per progress,
- crafted outputs,
- actions required,
- passive flag,
- knowledge visibility fields.

Source: `BitCraftServer/packages/game/src/messages/static_data.rs:941-967`.

The reducer validates tool, level, claim tech, and knowledge requirements before progressing a craft. The visible progress formula is tied to `actions_required * craft_count`, while progress added on a craft tick is capped by the remaining required action count (`BitCraftServer/packages/game/src/game/handlers/player_craft/craft.rs:461-468`, `BitCraftServer/packages/game/src/game/handlers/player_craft/craft.rs:480-493`).

No general crafting "success chance" was found in the reviewed craft reducer. The visible code path is requirement checks plus progress application. If requirements fail, progress is not applied or the action is cleared (`BitCraftServer/packages/game/src/game/handlers/player_craft/craft.rs:430-437`, `BitCraftServer/packages/game/src/game/handlers/player_craft/craft.rs:442-503`).

Resources, Gathering, And Extraction

Extraction recipes have consumed inputs, outputs, tool requirements, level requirements, XP per progress, and spawned placeable/resource fields. The exact schema is in `ExtractionRecipeDesc` in `static_data.rs`; the extraction reducer shows how rolls are applied.

Confirmed extraction behavior:

- Consumed item stacks can be removed probabilistically based on `consumption_chance` (`BitCraftServer/packages/game/src/game/handlers/player/extract.rs:348-365`).
- Tool durability can be reduced (`BitCraftServer/packages/game/src/game/handlers/player/extract.rs:367-369`).
- `base_damage` comes from tool power, while `damage` applies crit multiplier (`BitCraftServer/packages/game/src/game/handlers/player/extract.rs:373-389`).
- Extracted item stacks are rolled by `ProbabilisticItemStack::roll(ctx, damage_output)` (`BitCraftServer/packages/game/src/game/handlers/player/extract.rs:398-405`).
- Quest drops can roll during extraction (`BitCraftServer/packages/game/src/game/handlers/player/extract.rs:407-410`).
- XP is based on `experience_damage_output` and `experience_per_progress.quantity` (`BitCraftServer/packages/game/src/game/handlers/player/extract.rs:413-430`).
- Spawned placeable chance is scaled by `damage_output` and clamped between 0 and 1 (`BitCraftServer/packages/game/src/game/handlers/player/extract.rs:439-455`).

`ProbabilisticItemStack::roll` performs one random roll per damage/count unit and adds the stack quantity for each successful roll (`BitCraftServer/packages/game/src/game/entities/probabilistic_item_stack.rs:5-26`).

Resource deposits can spawn a follow-up resource on destruction if `on_destroy_yield_resource_id` is non-zero and the chance check passes (`BitCraftServer/packages/game/src/game/entities/resource_deposit.rs:211-225`).

Loot, Drops, And Rare Outcomes

The public repository confirms several loot/drop systems:

- `ContributionLootDesc` contains weighted/unweighted contribution loot metadata (`BitCraftServer/packages/game/src/messages/static_data.rs:1436-1442`).
- `LootTableDesc` and `LootRarityDesc` exist as static-data tables (`BitCraftServer/packages/game/src/messages/static_data.rs:1714-1727`).
- `QuestDropDesc` exists for quest-related drops (`BitCraftServer/packages/game/src/messages/static_data.rs:2600`).
- `ItemList::roll` sums configured probabilities, rolls against the total, and selects the first entry where the random value is consumed (`BitCraftServer/packages/game/src/game/entities/item_list.rs:61-78`).
- `PlaceableState::pick_growth_outcome` uses the same weighted-sum style for growth outcomes (`BitCraftServer/packages/game/src/game/entities/placeable_state.rs:123-145`).

Exact rare-drop percentages are not confirmed by the public repository alone unless the relevant live static-data rows are also available. The code shows the mechanics and probability fields, not the full production table contents.

Items, Cargo, Inventory, And Storage

Items and cargo are separate descriptor tables:

- `CargoDesc` includes id, name, description, volume, and knowledge metadata (`BitCraftServer/packages/game/src/messages/static_data.rs:666-675`).
- `ItemDesc` includes id, name, description, volume, durability, model/icon assets, tier, and tag (`BitCraftServer/packages/game/src/messages/static_data.rs:790-805`).

`InventoryState` stores pockets, inventory index, cargo index, owner entity, and player owner entity (`BitCraftServer/packages/game/src/messages/components.rs:640-651`). This is the core state shape used by many inventory, bank, storage, market, and deployable interactions.

Construction

Construction recipes are described by `ConstructionRecipeDesc`. The schema includes time, stamina, consumed building, required interior tier, level/tool requirements, consumed item/cargo stacks, consumed shards, XP per progress, required knowledge, required claim tech IDs, and action count (`BitCraftServer/packages/game/src/messages/static_data.rs:969-990`).

Construction and building placement check required claim technologies in multiple paths. For example, project placement and advancement check `required_claim_tech_ids` against `claim_tech_state` before allowing the action (`BitCraftServer/packages/game/src/game/handlers/buildings/project_site_place.rs:157-183`, `BitCraftServer/packages/game/src/game/handlers/buildings/project_site_advance_project.rs:158-194`).

Claim Research And Technology

Claim technology is represented by `ClaimTechDesc`, which includes supplies cost, research time, prerequisites, item inputs, member cap, area cap, supplies cap, XP-to-coin minting threshold, and recursively unlocked techs (`BitCraftServer/packages/game/src/messages/static_data.rs:2119-2135`).

Research can only be started by owner/co-owner players near the settlement building. The reducer checks prerequisites, consumes required items, ensures no current research is running, checks supply threshold protection, spends supplies, schedules a timer, and marks the tech as researching (`BitCraftServer/packages/game/src/game/handlers/claim/claim_tech_learn.rs:16-100`).

When the scheduled timer fires, `claim_tech_unlock_tech` clears the researching state and calls `unlock_claim_tech`. Unlocking appends the tech to `learned` and recursively unlocks anything in `tech_desc.unlocks_techs` (`BitCraftServer/packages/game/src/game/handlers/claim/claim_tech_unlock_tech.rs:23-77`).

Market Orders

Normal auction orders share the `AuctionListingState` structure for sell and buy order tables. It contains owner, claim, item id/type, price threshold, quantity, timestamp, and stored coins (`BitCraftServer/packages/game/src/messages/components.rs:1722-1742`).

Completed/cancelled/refund-style results are represented by `ClosedListingState`, which stores owner, claim, item stack, and timestamp (`BitCraftServer/packages/game/src/messages/components.rs:1744-1754`).

No claim-market tax or treasury fee was confirmed from the searched auction order state definitions and player trade handler search. The barter stall treasury code is confirmed separately and should not be treated as proof of general auction treasury income.

What This Means For Settlement Monitoring Apps

Useful confirmed data paths for an app:

- Claim treasury is real server state (`ClaimLocalState.treasury`).
- Crafting XP can mint claim treasury hex if the claim has learned tech with a positive `xp_to_mint_hex_coin`.
- Craft contribution/XP values are important because they can directly affect treasury minting.
- Claim members and permission flags are public state.
- Research state and learned claim tech are server state, but exact tech costs and thresholds require the static data records.
- Resource and loot systems are probabilistic, but exact drop rates require static-data rows and sometimes live state.

Important caution: BitJita may expose convenient API views, but the public server repo is the lower-level game logic. If BitJita data appears stale or incomplete, do not assume the server mechanics are wrong. Treat BitJita as a fan-made data provider layered on top of game/public data.